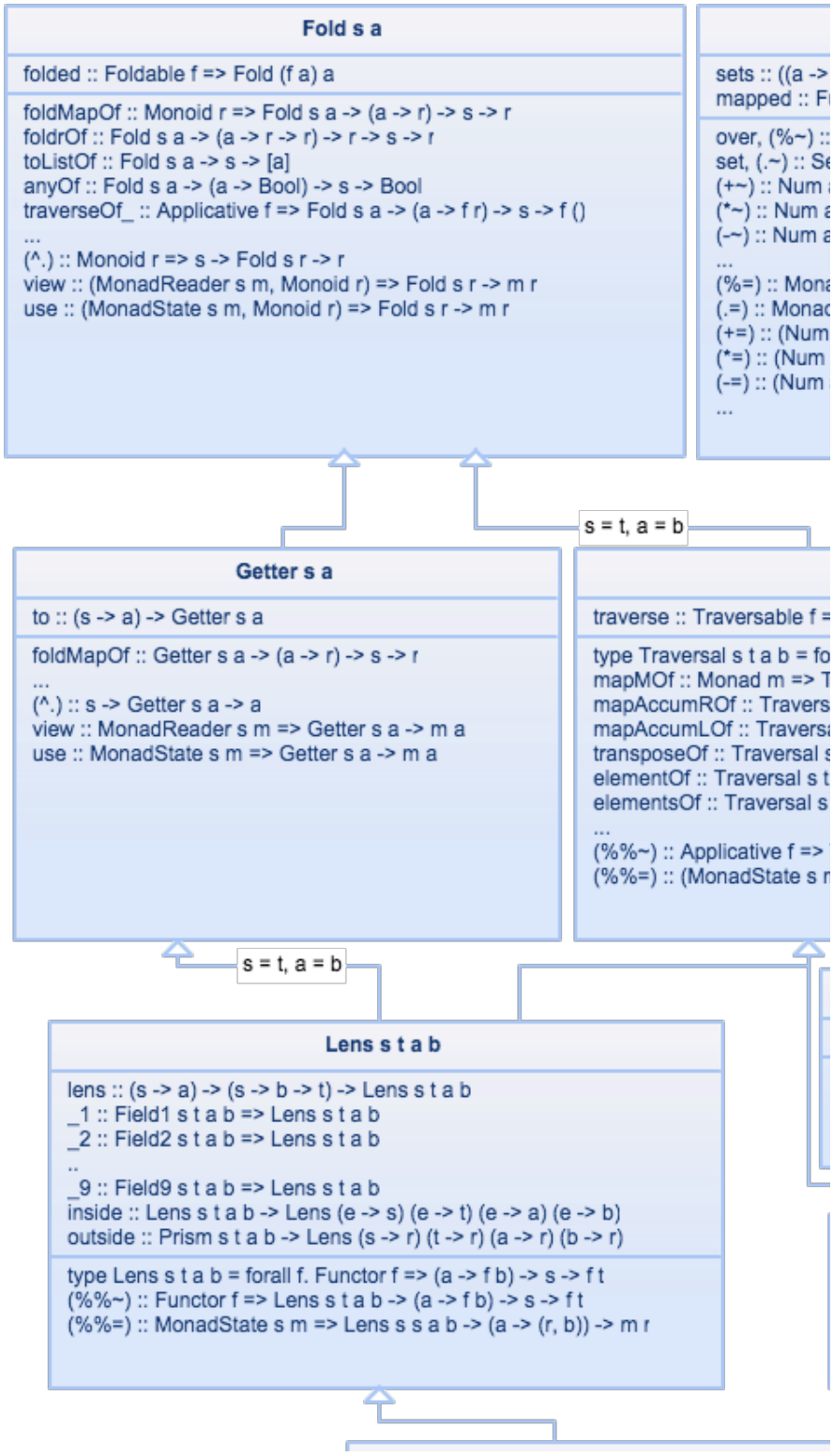
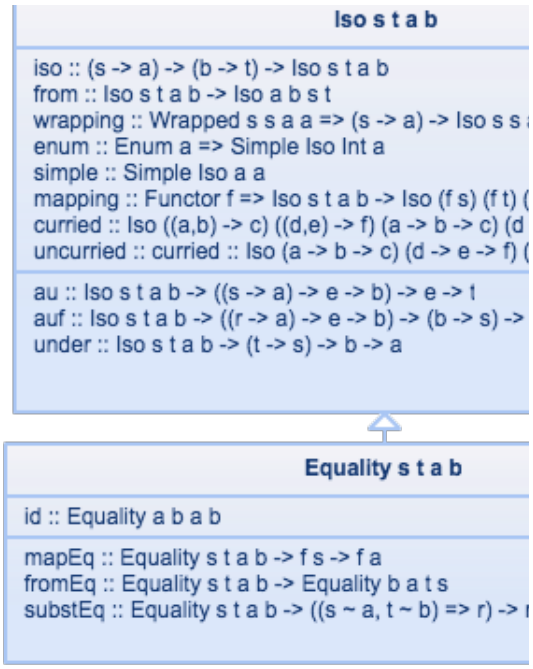
Resources
Haskell wiki - lenses: https://en.wikibooks.org/wiki/Haskell/Lenses_and_functional_references
Twan van Laarhoven lenses: https://www.twanvl.nl/blog/haskell/cps-functional-references

Modules
lens: https://hackage.haskell.org/package/lens
microlens: https://hackage.haskell.org/package/microlens





Iso ⊃ Lens ⊃ Getter ⊃ Fold
⊃ Traversal ⊃ Fold
⊃ Setter
⊃ Prism ⊃ Traversal ⊃ Fold
⊃ Setter

i.e.

```
-- Lens > Getter, Traversal, Fold, Setter
f = ...  :: Lens s t a b
f :: Lens s t a b                    -- OK
f :: Getter s t a b                  -- OK
f :: Traversal s t a b               -- OK
f :: Fold s a                        -- OK
f :: Setter s t a b                  -- OK

-- Traversal > Fold, Setter
f = ...  :: Traversal s t a b
f :: Lens s t a b                    -- ERROR
f :: Getter s t a b                  -- ERROR
f :: Traversal s t a b               -- OK
f :: Fold s a                        -- OK
f :: Setter s t a b                  -- OK

-- Getter > Fold
f = ...  :: Getter s t a b
f :: Lens s t a b                    -- ERROR
f :: Getter s t a b                  -- OK
f :: Traversal s t a b               -- ERROR
f :: Fold s a                        -- OK
f :: Setter s t a b                  -- ERROR

-- Fold
f = ...  :: Fold s a
f :: Lens s s a a                    -- ERROR
f :: Getter s s a a                  -- ERROR
```

```
f :: Traversal s s a a              -- ERROR
f :: Fold s a                       -- OK
f :: Setter s a a a                 -- ERROR

-- Setter
f = ... :: Setter s t a b
f :: Lens s t a b                   -- ERROR
f :: Getter s t a b                 -- ERROR
f :: Traversal s t a b              -- ERROR
f :: Fold s a                       -- ERROR
f :: Setter s t a b                 -- OK
```

Proof

```
-- Lens
lensIsTraversal :: Lens s t a b -> Traversal s t a b
lensIsTraversal = id
lensIsGetter :: Lens s t a b -> Getter s t a b
lensIsGetter = id
lensIsSetter :: Lens s t a b -> Setter s t a b
lensIsSetter = id
lensIsFold :: Lens s s a a -> Fold s a
lensIsFold = id

-- Traversal
traversalIsFold :: Traversal s s a a -> Fold s a
traversalIsFold = id
traversalIsSetter :: Traversal s t a b -> Setter s t a b
traversalIsSetter = id

-- Getter
getterIsFold :: Getter s s a a -> Fold s a
getterIsFold = id
```

Lens

Lens is a **functional reference** to some property a of object s i.e. property accessor as a function

Lens accesses a property by **traversing** over the object. Hence lens can be seen as a generalization of traversing

type Lens' s a = forall f. Functor f => (a -> f a) -> s -> f s
property a of object s

traverse :: (Traversable t, Applicative f) => (a -> f a) -> t a -> f (t a)
property a of (t a)

traverse is a specific instance of a lens

foldMap :: (Foldable t, Monoid m) => (a -> m) -> t a -> m
property a of (t a) summed as m

The action to perform on the property is specified by choosing a different **functor effect** f

e.g.

f = Const a getter s -> a
f = Identity setter s -> s

e.g. reference to (a, b) -> a

```
{-# LANGUAGE TupleSections #-}

-- reference to (a, b) -> a
_1 :: Len (a,b) (a',b) a a'
_1 f (a,b) = (,b) <$> f a


-- e.g. getter (a,b) -> a
-- with f = Const
(1, True) & _1 Const                -- Const a ~ a
(1, True) & _1 (Const . (*2))       -- Const a ~ a


-- e.g. setter (a,b) -> (g a, b)
-- with f = Identity
(1, True) & _1 (Identity . (*2))    -- Identity (2*a, b)
~ (2*a,b)
```

Main properties

Property a) Lenses are **composable**

e.g.

```
_1 :: Lens (a,b) (a',b) a a'                          =
forall f. Functor f => (a -> f a') -> (a, b) -> f (a',b)
_2 :: Lens (a,b) (a,b') b b'
   = forall f. Functor f => (b -> f b') -> (a,b) -> f
(a,b')
traverse :: (Traversable t, Applicative f)
  => (a -> f a) -> t a -> f (t a)

-- if t = []
_1 . traverse
  => (a -> f a') -> (a,b) -> f (a',b)
      . (u -> f u') -> ([u] -> f [u'])
  => (u -> f u') -> ([u],b) -> f ([u],b)
   with ([a] -> f [a']) = (u -> f u')
  => Lens ([u],b) ([u'],b) u u'
 each a of ([a],v)


_1 . traverse . _2
   => (u -> f u') -> ([u],b) -> f ([u],b)
      . (v -> f v') -> (c,v) -> f (c,v')
   => (v -> f v') -> ([(c,v)],b) -> f ([(c,v')],b)
with (t,a) -> f (t,a') = (u -> f u')
   => Lens ([(c,v)],b) ([(c,v')],b) v v'
 each a of ([(t,a)],v)
```

```
-- e.g.
let x = (
 [                    _1
  ("A", 1),           _1 . traverse = ("A", 1)
_1.traverse._1 = "A"  _1.traverse._2=1
  ("B", B)            _1 . traverse = ("B",
2)   _1.traverse._1 = "B"  _1.traverse._2=2
   ...
 ],
 True                 _2
)
x & view _1                              -- [("A",1),
("B",2), ... ]
x & view _2                              -- True
x & toListOf (_1 . traverse)             -- [("A",1),
("B",2), ... ]
x & toListOf (_1 . traverse . _1)        --
["A","B", ... ]
x & toListOf (_1 . traverse . _2)        --
[1,2, ... ]
```

Property b) allows **polymorphic update**
i.e. allow update that changes types
e.g.

```
-- monomorphic update i.e. no change in type
True & set id False                -- False
update Bool -> Bool
(True, 'a') & set _id False        -- (False, 'a')
 update (Bool, Char) -> (Bool, Char)

-- polymorphic update
True & set id ()                   -- ()
 update Bool -> ()
(True, 'a') & set _id ()           -- ((), 'a')
update (Bool, Char) -> ((), Char)
```

Types of effects

Effect	
(Contravariant f, Functor f)	Generalization of **Const** phantom x :: f a ~ Const r a
(Contravariant f, Applicative f)	Generalized **monoid** phantom (pure ()) :: f a ~ mempty x <*> y ~ x <> y
Applicative f	Generalized **weak monoid** Needs at least one term pure a :: f a ~ mempty x <*> y ~ x <> y

Effect types

Type	**Effect**	
Const r a	Contravariant, Functor	Value r
Monoid r => Const r a	Contravariant, Applicative	Sum of values
	e.g.	
	Const (Endo r)	Sequence of functions (r -> r) . (r -> r) . (r -> r)
	Const First	First value
	Const Any	x || x || ...
	Const All	x && x && ...
Identity a	Applicative, Settable	Function a -> a
... etc		

Types of lenses

	(a -> f b) -> s -> f t	
LensLike s t a b		
forall f. Functor f	=> (a -> f b) -> s -> f t	Lens s t a b
	=> (a -> f a) -> s -> f s	Lens' s a
forall f. Applicative	=> (a -> f b) -> s -> f t	Traversal s t a b
	=> (a -> f a) -> s -> f s	Traversal' s a
forall f. (Contravariant f, Functor f)	=> (a -> f b) -> s -> f t	Getter s t a b
	=> (a -> f a) -> s -> f s	Getter' s a
forall f. (Contravariant f, Applicative f)	=> (a -> f a) -> s -> f a	Fold s a
forall f. Settable f	=> (a -> f b) -> s -> f t	Setter s t a b
	=> (a -> f a) -> s -> f s	Setter' s a

==========

OLD

type Lens s t a b :: forall f. Functor f
=> (a -> f b) -> s -> f t

type Traversal s t a b = forall f. Applicative f
Generalization of traverse :: (Traversable t, Applicative f) => (a -> f b) -> t a -> f (t b)
=> (a -> f b) -> s -> f t
== traverse :: Traversal (t a) (t b) a b
type Traversal' s a = Traversal s s a a — Traverse with no type change

type Getter s t a b :: forall f. (Contravariant f, Functor f)
Generalized getter
=> (a -> f b) -> s -> f t

Contravariant + Functor ~ Const

e.g. f = Const a (a -> f a) -> s -> f s = (a -> Const a a) -> (s -> Const a s)

~ (s -> a) i.e. usual getter

type Fold s a = forall f. (Contravariant f, Applicative f) Fold as generalization of fold :: (Foldable t, Monoid m) => (a -> m) -> t a -> m
=> (a -> f a) -> s -> f s
~ (Contravariant f, Applicative f) => (a -> f a) -> s -> f s

Applicative ~ Semigroup

Applicative + Contravariant ~ Monoid

e.g. if f x = Const a x (a -> f a) -> [a] -> f [a] = (a -> Const a a) -> [a] -> Const a [a]

~ foldMap id [a]

= a <> a <> ...

e.g. if f x = Const All x (a -> f a) -> [a] -> f [a] = (a -> Const All a) -> [a] -> Const All [a]

~ foldMap All [Bool]

= All <> All <> ...

e.g. if f x = Const (Endo a) x (a -> f a) -> [a] -> f [a] = (a -> Const (Endo a) a) -> [a] -> Const (Endo a) [a]

~ foldMap (Endo) [a -> a]

~ (a -> a) <> (a -> a) ...

type Setter s t a b = forall f. Settable f Generalized setter
=> (a -> f b) -> s -> f t Settable f ~ Identity

e.g. f = Identity (a -> f a) -> s -> f t = (a -> Identity a) -> (s -> Identity s)

~ (a -> a) -> (s -> s) i.e. usual setter

Concrete instances

type Getting a s a :: (a -> Const a a) -> s -> Const a s Getter s -> a in CPS

Const a a ~ a

e.g. s = (a, x) ↦ a

type Getting r s a :: (a -> Const r a) -> s -> Const r s Getter s -> a -> r in CPS

Const r a ~ r

e.g. s = (a, x) ↦ a ↦ r

type ASetter s t a b = (a -> Identity b) -> s -> Identity t — Setter as generalization of fmap :: (a -> b) -> f a -> f b

~ (a -> b) -> s -> t

e.g. s = (a, x) ↦ t = (b, x)

type ASetter' s a = ASetter s s a a — Setter with no type change

e.g. s = (a, x) ↦ (a, x)

```
-- Traversal
f :: Traversal s t a b
f :: Fold (t a) a
f :: Traversable t => Setter s t a b

-- Getter
f :: Getter s t a b
f :: Fold s a


-- Lens
_1 :: Lens (a,c) (b,c) a b
_1 :: Getter (a,c) (b,c) a b
_1 :: Traversal (a,c) (b,c) a b
_1 :: Fold (a,c) a
_1 :: Setter (a,c) (b,c) a b

-- Getter
like 1 :: Getter s s Int Int
like 1 :: Fold s Int

-- Traversal
traverse :: Traversable t => Traversal (t a) (t b) a b
traverse :: Traversable t => Fold (t a) a
traverse :: Traversable t => Setter (t a) (t b) a b

-- Fold
folded :: Fold t => Fold (t a) a

-- Setter
mapped :: Functor f => Setter (f a) (f b) a b


-- Fold
toListOf :: (a -> Const (Endo [a])) -> s -> Const (Endo
[a]) s) -> s -> [a]
toListOf :: Fold s a -> s -> [a]
toListOf :: Traverse s s a a -> s -> [a]
toListOf :: Getter s s a a -> s -> [a]
toListOf :: Lens s s a a -> s -> [a]
```

```
-- Setter
set :: ((a -> Identity b) -> s -> Identity t) -> s -> t
set :: Setter s t a b -> s -> t
set :: Traverse s t a b -> s -> t
set :: Lens s t a b -> s -> t


-- Traverse
traverseOf :: ((a -> f b) -> s -> f t) -> (a -> f b) ->
s -> f t
traverseOf :: Applicative f => Traverse s t a b -> (a ->
f b) -> s -> f t
traverseOf :: Functor f => Lens s t a b -> (a -> f b) ->
s -> f t


-- Getter
view :: MonadReader s m => ((a -> Const a a) -> s ->
Const a s) -> s -> a
view :: MonadReader s m => Getter s a -> s -> a
view :: MonadReader s m => Lens s a -> s -> a
```